1.yes — Generate repeated text

Attribute	Meaning
yes "text"	Repeatedly print the specified text
yes	Repeatedly print y
Ctrl + C	Stop continuous output

```
vanam@Uday:~$ yes "Uday"
```

```
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
Uday
```

```
vanam@Uday:~$ yes "Linux Practice" | head -n 5
Linux Practice
Linux Practice
Linux Practice
Linux Practice
Linux Practice
```

2. tty — Identify your current terminal

Attribute	Meaning
tty	Display the current terminal device
tty -s	Check whether standard input is a terminal without printing the device name

3**. comm — Compare two sorted files**

```
vanam@Uday:~$ printf 'Java\nPython\nSQL\n' > file1
printf 'Java\nLinux\nPython\n' > file2
comm file1 file2
                Java
        Linux
                Python
SQL
```

4. split — Break a file into smaller files

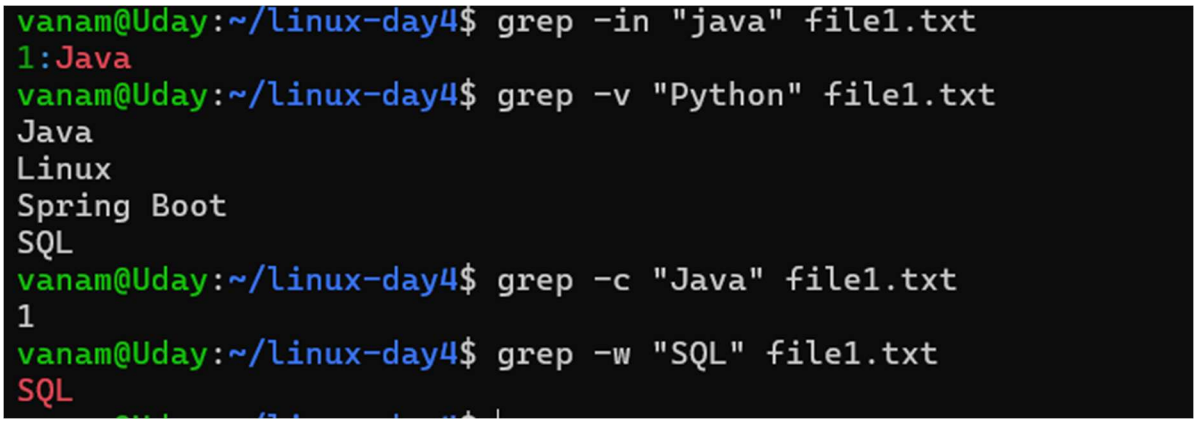
Options:

- **-l 2 — put two lines in each output file.**
- **-b 1K — split into chunks of approximately 1 KB.**
- **-d — use numeric suffixes.**

```
vanam@Uday:~$ seq 1 10 > numbers.txt
split -l 3 -d numbers.txt chunk_
ls chunk_*
cat chunk_00
chunk_00  chunk_01  chunk_02  chunk_03
1
2
3
```

5. grep — Search for matching text

- `-i` — Ignore uppercase/lowercase differences.
- `-n` — Show line numbers.
- `-v` — Show lines that do not match.
- `-c` — Count matching lines.
- `-w` — Match a whole word.



6. sed — Edit or filter text

- `s/old/new/` — Replace the first occurrence on each line.
- `g` — Replace all occurrences on each line.
- `-n` — Suppress automatic printing.
- `p` — Print selected lines.



```
vanam@Uday:~/linux-day4$ sed -n '2,4p' file1.txt
Python
Linux
Spring Boot
```

```
vanam@Uday:~/linux-day4$ sed 's/Java/Advanced Java/g' file1.txt
Advanced Java
Python
Linux
Spring Boot
SQL
```

7. awk — Extract and process text

awk processes text by records and fields. It is useful for logs, CSV-like files, and command output.

- `$1` — First field.
- `$2` — Second field.
- `$NF` — Last field.
- `NF` — Number of fields in the current line.
- `NR` — Current input line number.
- `-F,` — Use a comma as the field separator.

```
vanam@Uday:~/linux-day4$ awk '{print $1}' file1.txt
Java
Python
Linux
Spring
SQL
```

```
vanam@Uday:~/linux-day4$ awk '{print NR, $1}' file1.txt
1 Java
2 Python
3 Linux
4 Spring
5 SQL
```

```
vanam@Uday:~/linux-day4$ awk '{print $NF}' file1.txt
Java
Python
Linux
Boot
SQL
```

8. sort — Sort lines of text

- `-r` — Reverse the sort order.
- `-n` — Sort numerically.
- `-u` — Remove duplicate lines from the sorted output.
- `-f` — Ignore case while sorting.
- `-o file` — Write output to a specified file.

```
vanam@Uday:~/linux-day4$ sort file1.txt
Java
Linux
Python
SQL
Spring Boot
```

```
vanam@Uday:~/linux-day4$ sort -r file1.txt
Spring Boot
SQL
Python
Linux
Java
```

```
vanam@Uday:~/linux-day4$ sort -f file1.txt
Java
Linux
Python
Spring Boot
SQL
```

9.comm — Find common and unique lines

Compares two sorted files and displays three columns:

1. Lines only in the first file.
2. Lines only in the second file.
3. Lines common to both files.

```
vanam@Uday:~/linux-day4$ comm file1.txt file2.txt
        Docker
        Git
                Java
Linux
                Python
SQL
                Spring Boot
```

- `-1` — Hide lines unique to the first file.
- `-2` — Hide lines unique to the second file.
- `-3` — Hide lines common to both files.

```
vanam@Uday:~/linux-day4$ comm -12 file1.txt file2.txt
Java
Python
Spring Boot
```

10. tr — Transform characters

```
vanam@Uday:~/linux-day4$ cat file1.txt | tr 'a-z' 'A-Z'
JAVA
LINUX
PYTHON
SQL
SPRING BOOT
```